자바 표준 인터페이스 활용하기 - Comparable과 Comparator

목차

1. 학습 목표
2. 정렬은 왜 인터페이스로 만들어져 있을까
3. Comparable - 객체 스스로 기본 정렬 기준을 가진다
4. Comparator - 정렬 기준을 외부에서 주입한다
5. Comparator를 익명 객체로 구현하기
6. Comparable과 Comparator 비교
7. 참고 - 람다식으로도 작성 가능
8. 핵심 정리

---

1. 학습 목표

이 교안을 마치면 다음을 할 수 있다.

- 자바 표준 인터페이스인 Comparable과 Comparator의 역할을 설명할 수 있다.
- `compareTo()`와 `compare()` 메소드의 반환값이 정렬 순서를 어떻게 결정하는지 이해한다.
- 클래스에 Comparable을 구현해서 기본 정렬 기준(natural ordering)을 정의할 수 있다.
- Comparator를 외부 클래스 또는 익명 객체로 구현해서 정렬 기준을 자유롭게 바꿀 수 있다.

---

2. 정렬은 왜 인터페이스로 만들어져 있을까

자바에는 리스트나 배열을 정렬해 주는 기능이 이미 들어 있다.

```java
Collections.sort(list);
```

그런데 자바 입장에서 보면 한 가지 문제가 있다. 리스트 안에 들어있는 객체가 `Student`인지 `Book`인지 자바는 알 수 없다. 그러니 두 객체 중 누가 더 "큰지" 자바가 스스로 판단할 방법이 없다.

이 문제를 해결하기 위해 자바는 두 개의 표준 인터페이스를 만들어 두었다.

- **Comparable:** 객체 스스로 "나는 이렇게 비교한다"라고 비교 규칙을 정의하는 인터페이스
- **Comparator:** 객체 외부에서 "이 객체들은 이런 기준으로 비교해라"라고 정렬 기준을 따로 정의하는 인터페이스

`Collections.sort()`는 이 두 인터페이스 중 하나를 사용해서 정렬을 수행한다.

> 공식 문서: Collections.sort (Java SE 17)

---

3. Comparable - 객체 스스로 기본 정렬 기준을 가진다

3-1. Comparable이란

`Comparable`은 `java.lang` 패키지에 있는 표준 인터페이스이다. 클래스가 이 인터페이스를 구현하면, 그 클래스의 객체끼리 **순서 비교(ordering)**가 가능해진다. 즉 두 객체 중 어느 쪽이 앞에 오고 어느 쪽이 뒤에 와야 하는지 정할 수 있게 된다. 이 비교 규칙이 곧 그 클래스의 기본 정렬 기준(natural ordering)이 된다.

핵심 메소드는 단 하나, `compareTo()`이다.

```java
public int compareTo(T other)
```

반환값의 의미는 다음과 같다.

- **음수 반환**: 현재 객체(this)가 매개변수 객체(other)보다 작다 → this가 앞에 위치
- **0 반환**: 두 객체가 같다
- **양수 반환**: 현재 객체(this)가 매개변수 객체(other)보다 크다 → this가 뒤에 위치

> 공식 문서: Comparable (Java SE 17)

3-2. Student 클래스에 Comparable 구현하기

학생을 점수 기준으로 정렬하고 싶다고 해보자. 그러면 `Student` 클래스가 직접 "나는 점수로 비교한다"라고 정의해 주면 된다.

```java
```

```java
public class Student implements Comparable<Student> {

    private String name;
    private int    score;

    public Student(String name, int score) {
        this.name  = name;
        this.score = score;
    }

    public String getName()  { return name; }
    public int    getScore() { return score; }

    // 점수 오름차순 기준
    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.score, other.score);
    }

    @Override
    public String toString() {
        return name + " (" + score + "점)";
    }
}
```

`Integer.compare(this.score, other.score)`의 반환값은 다음과 같이 결정된다.

- this의 점수가 더 작으면 → 음수 반환 → this가 앞으로 (낮은 점수가 앞)
- this의 점수가 더 크면 → 양수 반환 → this가 뒤로 (높은 점수가 뒤)

결과적으로 점수가 낮은 학생부터 높은 학생 순으로 정렬된다.

> 공식 문서: Integer.compare (Java SE 17)

3-3. Comparable을 구현했더니 정렬이 자동으로 된다

```java
```

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class ComparableExample {
    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();
        students.add(new Student("홍길동", 85));
        students.add(new Student("김길동", 92));
        students.add(new Student("이길동", 78));
        students.add(new Student("박길동", 95));
        students.add(new Student("최길동", 88));

        System.out.println("=== 정렬 전 ===");
        for (Student s : students) {
            System.out.println(s);
        }

        // Student.compareTo() 가 자동으로 호출됨
        Collections.sort(students);

        System.out.println("\n=== 점수 오름차순 정렬 후 ===");
        for (Student s : students) {
            System.out.println(s);
        }
    }
}
```

실행 결과는 다음과 같다.

```
=== 정렬 전 ===
홍길동 (85점)
김길동 (92점)
이길동 (78점)
박길동 (95점)
최길동 (88점)

=== 점수 오름차순 정렬 후 ===
이길동 (78점)
홍길동 (85점)
최길동 (88점)
김길동 (92점)
박길동 (95점)
```

`Collections.sort(students)`만 호출했을 뿐인데 정렬이 된다. 이는 `Student` 클래스 내부에 이미 비교 규칙(`compareTo`)이 정의되어 있기 때문이다.

3-4. 내림차순으로 바꾸려면

`Integer.compare()`에 넘기는 두 값의 순서만 바꾸면 된다.

```java
@Override
public int compareTo(Student other) {
    return Integer.compare(other.score, this.score);   // 점수 내림차순
}
```

이제 점수가 높은 학생이 앞에 온다.

---

4. Comparator - 정렬 기준을 외부에서 주입한다

4-1. Comparable의 한계

Comparable은 클래스의 기본 정렬 기준(natural ordering)을 하나 정의한다. 여러 정렬 기준이 필요하면 Comparator를 사용한다.

예를 들어 `Student`를 어떨 때는 점수순으로, 어떨 때는 이름순으로 정렬하고 싶다면 매번 `Student` 클래스 안의 `compareTo`를 고칠 수는 없다. 이럴 때 사용하는 것이 `Comparator`이다.

4-2. Comparator란

`Comparator`는 `java.util` 패키지에 있는 표준 인터페이스이다. **객체 외부에서** 정렬 기준을 정의하는 데 사용한다.

핵심 메소드는 `compare()`이다.

```java
public int compare(T o1, T o2)
```

반환값 규칙은 Comparable과 동일하다.

- **음수 반환**: o1이 o2보다 앞에 위치
- **0 반환**: 두 값이 같다
- **양수 반환**: o1이 o2보다 뒤에 위치

> 공식 문서: Comparator (Java SE 17)

4-3. Collections.sort()의 두 가지 형태

`Collections.sort()`는 두 가지 형태로 호출할 수 있다. 어떤 인터페이스를 사용하느냐에 따라 형태가

달라진다.

형태	사용 인터페이스	의미
`Collections.sort(list)`	Comparable	객체가 가진 기본 정렬 기준(natural ordering)으로 정렬
`Collections.sort(list, comparator)`	Comparator	외부에서 주입한 **별도의 기준**으로 정렬

첫 번째 형태는 리스트 안의 객체가 Comparable을 구현하고 있어야 동작한다. 두 번째 형태는 두 번째 인자로 Comparator를 직접 넘겨주기 때문에, 리스트 안의 객체가 Comparable을 구현하지 않아도 정렬할 수 있다.

> 공식 문서: Collections.sort(List, Comparator) (Java SE 17)

4-4. Comparator 구현 클래스 만들기

문자열을 내림차순으로 정렬하는 Comparator를 만들어 본다.

```java
import java.util.Comparator;

class DescendingComparator implements Comparator<String> {

    @Override
    public int compare(String a, String b) {
        return b.compareTo(a);    // 내림차순
    }
}
```

`b.compareTo(a)`는 평소와 비교 순서를 뒤집은 것이다. 이렇게 하면 "큰 값이 앞으로 오는" 정렬이 된다.

> 참고: `String` 클래스는 이미 Comparable을 구현하고 있어서 `compareTo()`를 사용할 수 있다.

4-5. Collections.sort()에 Comparator 전달하기

위에서 만든 Comparator 구현 클래스를 사용해서 정렬해 본다.

```java
```

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class ComparatorExample {

    public static void main(String[] args) {

        List<String> list = new ArrayList<>();
        list.add("banana");
        list.add("pineapple");
        list.add("grape");
        list.add("orange");

        System.out.println("정렬 전: " + list);

        // 1) 기본 정렬 - String의 Comparable(compareTo)을 사용 → 오름차순
        Collections.sort(list);
        System.out.println("오름차순 정렬 후: " + list);

        // 2) 외부 Comparator를 전달 → 내림차순
        Collections.sort(list, new DescendingComparator());
        System.out.println("내림차순 정렬 후: " + list);
    }
}
```

같은 리스트에 대해 정렬 기준을 자유롭게 갈아 끼울 수 있다는 점이 핵심이다.

---

5. Comparator를 익명 객체로 구현하기

5-1. 매번 클래스를 만드는 게 번거롭다

앞에서 본 `DescendingComparator`처럼, Comparator를 **딱 한 번만 사용**하는 경우 클래스를 따로 만드는 것이 번거롭다.

이럴 때는 클래스를 만들지 않고, **그 자리에서 바로** 인터페이스를 구현한 객체를 만들 수 있다. 이를 **익명 구현 객체**라고 한다.

5-2. 익명 객체 문법

```java
```

```
new 인터페이스이름() {
    @Override
    public 반환타입 메소드이름(...) {
        // 구현 내용
    }
};
```

`new` 뒤에 인터페이스 이름이 오고, 중괄호 안에 메소드 구현을 바로 작성한다. 즉 "이름 없는 클래스"를 만들면서 동시에 객체로 만드는 문법이다.

5-3. Comparator를 익명 객체로 - 내림차순

```java
Collections.sort(list, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return b.compareTo(a);   // 내림차순
    }
});

System.out.println("내림차순 정렬 후 (익명 객체): " + list);
```

`DescendingComparator` 클래스를 만들지 않고도 동일한 동작을 했다.

5-4. 익명 객체로 - 문자열 길이 기준 정렬

정렬 기준을 만들 때는 두 가지 단계로 생각하면 된다.

1. 어떤 기준으로 정렬할지 정한다.
2. `compare` 안에 그 조건을 넣는다.

문자열의 길이를 기준으로 짧은 것부터 정렬해 본다.

```java
Collections.sort(list, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return Integer.compare(a.length(), b.length());   // 길이 오름차순
    }
});

System.out.println("길이 오름차순: " + list);
```

`Integer.compare(a.length(), b.length())`의 결과가 음수면 a가 더 짧다는 뜻이고, 그러면 a가 앞에 온다.

> **참고: 왜 `a.length() - b.length()`처럼 빼지 않을까?**
> 간단해 보여서 뺄셈 방식을 자주 쓰지만, 두 값의 차이가 너무 크면 int 범위를 벗어나서 결과가 뒤집히는 **오버플로(overflow)** 문제가 발생할 수 있다. `Integer.compare()`는 이 문제 없이 안전하게 비교한다. 다른 교재나 코드에서 뺄셈 방식을 보더라도, 새로 작성할 때는 `Integer.compare()`를 사용하는 것이 안전하다.
> 실제로 어떻게 뒤집히는지 보자.
>
> ```java
> int a = Integer.MAX_VALUE;    // 2147483647
> int b = -1;
>
> // a가 b보다 크니까 양수가 나와야 정상
> System.out.println(a - b);                  // -2147483648 (음수! 결과가 뒤집힘)
> System.out.println(Integer.compare(a, b));  // 1 (정상)
> ```
>
> `a - b`는 수학적으로는 `2147483648`이 되어야 하지만, int가 표현할 수 있는 최댓값은 `2147483647`이다. 한 칸을 넘어가면서 값이 한 바퀴 돌아 음수로 떨어진다. 이 결과가 정렬 함수에 전달되면 "a가 b보다 작다"고 판단해서 정렬 순서가 뒤집힌다. `Integer.compare()`는 내부에서 단순히 크기만 비교하기 때문에 이런 문제가 없다.

5-5. 익명 객체로 - 다중 정렬 기준

"길이 기준으로 정렬하되, 길이가 같으면 사전순으로" 같이 기준이 두 개일 때도 익명 객체로 표현할 수 있다.

```java
Collections.sort(list, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {

        // 1차 기준: 문자열 길이
        if (a.length() != b.length()) {
            return Integer.compare(a.length(), b.length());
        }

        // 2차 기준: 길이가 같으면 사전순
        return a.compareTo(b);
    }
});

System.out.println("길이 → 사전순: " + list);
```

`compare()` 메소드 안에서 조건을 분기해서 1차/2차 기준을 차례로 적용한다.

---

6. Comparable과 Comparator 비교

구분	Comparable	Comparator
패키지	`java.lang`	`java.util`
핵심 메소드	`compareTo(T other)`	`compare(T o1, T o2)`
정렬 기준 위치	객체 **내부**	객체 **외부**
정렬 기준 개수	보통 한 가지 (기본 정렬 기준)	자유롭게 여러 개
사용 시점	클래스의 기본 정렬 기준(natural ordering)을 정할 때	상황에 따라 **다양한 정렬**이 필요할 때

> **권장 사용 방식**
> 클래스의 기본 정렬 기준(natural ordering)은 Comparable로 정의하고, **추가 정렬 기준**이 필요할 때마다 Comparator를 사용한다.

> 공식 문서: Comparable (Java SE 17) 공식 문서: Comparator (Java SE 17)

---

7. 참고 - 람다식으로도 작성 가능

Comparator처럼 메소드가 하나뿐인 인터페이스는 **람다식**으로 더 짧게 작성할 수도 있다. 지금 단계에서는 "이런 표현도 있구나" 정도로만 알아두면 된다.

```java
// 익명 객체
Collections.sort(list, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return b.compareTo(a);
    }
});

// 람다식 - 위와 똑같은 동작
Collections.sort(list, (a, b) -> b.compareTo(a));
```

```java
// 길이 기준 정렬도 람다로
Collections.sort(list, (a, b) -> Integer.compare(a.length(), b.length()));
```

---

8. 핵심 정리

1. **Comparable**은 객체 자신이 비교 규칙을 가진다. 클래스 안에 `compareTo()`를 구현한다.
2. **Comparator**는 외부에서 정렬 기준을 주입한다. `compare()`를 구현한다.
3. 두 메소드 모두 반환값 규칙은 같다.
   - 음수 → 앞 인자가 앞에 위치
   - 0 → 같음
   - 양수 → 앞 인자가 뒤에 위치
4. `Collections.sort(list)`는 Comparable을, `Collections.sort(list, comparator)`는 Comparator를 사용한다.
5. 정수 비교는 `a - b`처럼 빼지 말고 `Integer.compare(a, b)`를 사용한다. (오버플로 방지)
6. 한 번만 사용할 Comparator는 클래스로 만들지 않고 **익명 객체**로 그 자리에서 구현하는 것이 편리하다.
7. 클래스의 기본 정렬 기준(natural ordering)은 Comparable로 정의하고, **추가 정렬 기준**이 필요할 때마다 Comparator를 사용한다.
8. **Comparable**을 구현할 때는 `compareTo()`의 결과가 `equals()`와 일관되도록 설계하는 것이 권장된다. 즉 두 객체가 정렬 기준으로 같다면(`compareTo == 0`) `equals`도 `true`가 되도록 맞추는 것이 안전하다.